---------- Slip 3 ----------

Q.1) Print the type of file and inode number where file name accepted through Command Line

```
#include <stdio.h>
#include <sys/stat.h>

int main(int argc, char *argv[]) {

   if (argc != 2) {
      printf("Usage: %s <filename>\n", argv[0]);
      return 1;
   }

   struct stat fileStat;

   if (stat(argv[1], &fileStat) == -1) {
      perror("stat");
      return 1;
   }

   // Print inode number
   printf("Inode Number: %lu\n", fileStat.st_ino);

   // Print file type
   printf("File Type: ");
   if (S_ISREG(fileStat.st_mode))
      printf("Regular File\n");
   else if (S_ISDIR(fileStat.st_mode))
      printf("Directory\n");
   else if (S_ISCHR(fileStat.st_mode))
      printf("Character Device\n");
   else if (S_ISBLK(fileStat.st_mode))
      printf("Block Device\n");
   else if (S_ISFIFO(fileStat.st_mode))
      printf("FIFO/Named Pipe\n");
```

```
    else if (S_ISLNK(fileStat.st_mode))
        printf("Symbolic Link\n");
    else if (S_ISSOCK(fileStat.st_mode))
        printf("Socket\n");
    else
        printf("Unknown Type\n");

    return 0;
}
```

Q.2) Write a C program which creates a child process to run linux/ unix command or any user defined program. The parent process set the signal handler for death of child signal and Alarm signal. If a child process does not complete its execution in 5 second then parent process kills child process.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child_pid;   // Global so signal handlers can access it

void child_death_handler(int sig) {
    printf("\nChild process finished execution.\n");
}

void alarm_handler(int sig) {
    printf("\nChild process did not finish in 5 seconds. Killing child...\n");
    kill(child_pid, SIGKILL);
}

int main() {

    // Set signal handlers in parent
    signal(SIGCHLD, child_death_handler);
```

```
    signal(SIGALRM, alarm_handler);

    child_pid = fork();

    if (child_pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (child_pid == 0) {
        // -------------------------
        // Child Process
        // -------------------------
        printf("Child running command...\n");

        // Example: Run "sleep 10" or replace with any command
        execlp("sleep", "sleep", "10", NULL);

        perror("exec failed");
        exit(1);
    }

    // -------------------------
    // Parent Process
    // -------------------------
    printf("Parent: child started (PID = %d)\n", child_pid);

    // Set 5-second timer
    alarm(5);
    // Wait for child (blocking)
    wait(NULL);
    printf("Parent: exiting.\n");
    return 0;
}
```